

Tests et preuve de programmes

I) Tri par insertion

Je vous donne le code du tri par insertion :

```
def insere(L_triee, x):
    """ Cette fonction prend en argument une liste triée (en ordre croissant)
    d'entiers, et un élément x, et l'insère en place dans la liste. """
    for i in range(len(L_triee)):
        if x < L_triee[i]:
            x, L_triee[i] = L_triee[i], x #inverse x et L[i] si ils sont dans le
désordre
            #à la fin, l'élément restant est le plus grand de la liste
    L_triee.append(x)

def tri_insertion(L):
    """ Cette fonction prend en argument une liste d'entier, et renvoie une nouvelle
    liste triée en ordre croissant avec l'algorithme du tri par insertion. """
    L_triee = []
    for i in range(len(L)):
        insere(L_triee, L[i])
    return L_triee
```

Question 1

Proposez des tests aux cas limites, pour :

- a) la fonction insere
- b) la fonction tri_insertion

Pour les deux fonctions, on veut en particulier tester le cas où la liste est vide comme cas limite.

```
L = []
insere(L, 3)
assert L == [3] #ici le 3 pourrait être n'importe quoi
```

```
assert tri_insertion([]) == []
assert tri_insertion([3]) == [3]
```

Question 2

Donnez une pre-condition, un invariant, et une post-condition pour la fonction insere. En déduire sa correction partielle.

Preuve de correction partielle de la fonction insere.

Version (trop) détaillée (à noter que ce genre de preuves est très fastidieuse à vraiment formaliser, donc je vous donne une version plus rapide après) :

On note les variables à la fin du ième tour de boucle avec un indice L_triee_i .

On s'intéresse à la boucle :

Pré-condition : L_triee est triée.

Invariant au tour de boucle i : L_triee_i est triée, x_i est plus grand que tous les éléments de $L_triee_i[0 : (i + 1)]$, et L_triee_i et x_i contiennent (à eux deux) exactement les éléments donnés en argument.

Preuve de l'invariant : Avant la boucle, l'invariant est vérifié (c'est la pré-condition).

Supposons que l'invariant soit vérifié avant le tour de boucle i .

On a donc L_triee_{i-1} qui est triée, x_{i-1} est plus grand que tous les éléments de $L_triee_{i-1}[0 : i]$, et $L_triee_{i-1}[0 : i]$ et x_{i-1} contiennent (à eux deux) exactement les éléments donnés en argument.

NB : on peut avoir $i = 0$, auquel cas $L_triee_{i-1}[0 : i]$ est vide.

Alors :

- si $x_{i-1} < L_triee_{i-1}[i]$, on les inverse, et donc, après la boucle,
 - Comme x_{i-1} était plus grand que les éléments de $L_triee_{i-1}[0 : i]$, $L_triee_i[i] = x_{i-1}$ l'est aussi. De plus, L_triee_{i-1} était triée, donc $L_triee_{i-1}[i]$ est plus petit que les éléments de $L_triee_i[(i + 1) :]$ et donc $L_triee_i[i]$ l'est aussi et L_triee_i est triée.
 - Comme x_{i-1} était plus grand que les éléments de $L_triee_{i-1}[0 : i]$, et que $x_{i-1} < L_triee_{i-1}[i]$, $x_i = L_triee_{i-1}[i]$ est plus grand que tous les éléments de $L_triee_{i-1}[0 : (i + 1)]$
- sinon, ils ne sont pas inversés, et L_triee reste triée et x reste plus grand que le début de la liste.

On en déduit que l'invariant au tour de boucle i implique celui au tour de boucle $i + 1$, et donc l'invariant est vrai.

Post-condition : On applique l'invariant au dernier tour de boucle, soit $i = \text{len}(L) - 1$. On a, après la boucle :

- L_triee est triée.
- L_triee et x contiennent (à eux deux) exactement les éléments donnés en argument.
- x est plus grand que tous les éléments de L_triee .

Donc, en ajoutant x à la fin de L_triee , on obtient bien le résultat voulu.

Version normale

On s'intéresse à la boucle :

Pré-condition : L_triee est triée.

Invariant au tour de boucle i : L_triee est triée, x est plus grand que tous les éléments de $L_triee[0 : (i + 1)]$.

Preuve de l'invariant : Avant la boucle, l'invariant est vérifié (c'est la pré-condition).

Alors :

- si $x < L_triee[i]$, on les inverse, et donc, après la boucle,
 - Comme x était plus grand que les éléments de $L_triee[0 : i]$, $L_triee[i]$ le sera aussi après échange. De plus, L_triee était triée, donc $L_triee[i]$ était plus petit que les éléments de $L_triee[(i + 1) :]$ et x l'est aussi. On en déduit que L_triee reste triée.
 - Comme x était plus grand que les éléments de $L_triee[0 : i]$, et que $x < L_triee[i]$, x reste plus grand que tous les éléments de $L_triee[0 : (i + 1)]$
- sinon, ils ne sont pas inversés, et L_triee reste triée et x reste plus grand que le début de la liste.

Post-condition : On applique l'invariant au dernier tour de boucle, soit $i = \text{len}(L) - 1$. On a, après la boucle :

- L_triee est triée.
- L_triee et x contiennent (à eux deux) exactement les éléments donnés en argument.
- x est plus grand que tous les éléments de L_triee .

Donc, en ajoutant x à la fin de L_triee , on obtient bien le résultat voulu.

Ici, la terminaison des deux fonctions est évidente, car elles ne contiennent ni appels récursifs, ni boucles while

Question 3

En supposant la fonction `insere` correcte, donnez une pre-condition, un invariant, et une post-condition pour la fonction `tri_insertion`. En déduire sa correction.

Preuve de correction partielle de la fonction `tri_insertion`

On suppose que `insere` est correcte.

Pre-condition

`L_triee = []` est triée

Invariant

`L_triee` est triée et contient les éléments de `L[0:i]`.

Preuve de l'invariant

Si au début d'un tour de boucle, l'invariant est respecté, alors, en appliquant `insere(L_triee, L[i])`, `L_triee` contient bien les éléments de `L[0:(i+1)]` et est triée, par correction de `insere`.

Post-condition

On en déduit que `L_triee` est bien `L`, triée. Donc la fonction est correcte.

II) Exponentiation rapide

```
def exp_rapide(x, n):
    """ Pour n un entier positif et x un nombre non nul (entier ou flottant), calcule
    l'exponentielle de x par n en O(log(n)) opérations.
    """
    resultat = 1
    while n > 0:
        # si n est pair, on factorise un 2 dans le calcul : x ^ n = (x ^ 2) ^ (n / 2)
        if n % 2 == 0:
            x *= x
            n //= 2
        # sinon, on multiplie le résultat par x, pour enlever 1 à n.
        else:
            resultat *= x
            n -= 1
    return resultat
```

Question 4

Proposez un partitionnement de l'espace d'entrée, et des tests aux cas limites, pour la fonction `exp_rapide`.

Partitionnement de l'espace d'entrée :

- `x` négatif, `x` positif
- `x` flottant, `x` entier

Tests aux cas limites : `n = 0`

```
assert exp_rapide(2, 0) == 1
assert exp_rapide(-2, 0) == 1
```

Question 5

Donnez un variant de boucle pour la fonction `exp_rapide`. En déduire sa terminaison.

Variante de boucle : n.

A chaque tour de boucle, n décroît strictement. De plus, n est un entier positif. Donc la fonction termine.

Question 6

Donnez une pré-condition, un invariant, et une post-condition pour la fonction `exp_rapide`. En déduire sa correction.

Pré-condition

`resultat = 1`

Invariant

En notant x_0 et n_0 les arguments, à la fin de chaque tour de boucle, on a : $\text{resultat} * x^n = x_0^{n_0}$

Preuve de l'invariant

- L'invariant est vérifié avant la boucle (pré-condition).
- Supposons l'invariant vérifié avant un tour de boucle. Alors, on a deux cas :
 - n est pair, auquel cas on a bien $(x^2)^{\frac{n}{2}} = x^n$.
 - n est impair, auquel cas on a bien $x * x^{n-1} = x^n$.

Donc l'invariant reste vrai.

Post-condition

On applique l'invariant à la sortie de la boucle. On a donc, comme n est un entier positif qui ne vérifie pas la condition $n > 0$, $n = 0$.

Donc on a : $x_0^{n_0} = \text{resultat} * x^0 = \text{resultat}$, et la fonction est correcte.

III) Tri rapide

```
def tri_rapide(L):
    """ Pour L une liste, renvoie une nouvelle liste triée par ordre croissant
    contenant les mêmes éléments.
    """
    if (len(L) <= 1):
        return L[:]
    else:
        pivot = L[0]
        #separe la liste en deux : les éléments plus grand et plus petits que le
        pivot
        plus_petits = [el if el <= pivot for el in L[1:]]
        plus_grands = [el if el > pivot for el in L]

        #on trie ces deux listes en appelant récursivement notre fonction tri_rapide
        plus_petits_triee = tri_rapide(plus_petits)
        plus_grands_triee = tri_rapide(plus_grands)

        #enfin, on concatène ces listes pour obtenir le résultat
        return plus_petits_triee + [pivot] + plus_grands_triee
```

Question 7

Les tests de la fonction `tri_insertion` sont-ils aussi valides pour `tri_rapide` ?

Oui, c'est un algorithme de tri.

Question 8

Donnez un variant pour la fonction `tri_rapide`, c'est-à-dire une quantité qui décroît à chaque appel récursif. En déduire sa terminaison.

Variant (pas de boucle du coup) :

$\text{len}(L)$ décroît strictement à chaque appel récursif, et est un entier positif.

Preuve du variant

A chaque appel, ni la liste `plus_petit` (extraite de `L[1:]`), ni la liste `plus_grand` (qui ne contient aucun élément égal à `pivot`), ne peuvent contenir le `pivot`. Donc elles sont nécessairement de longueur au plus $\text{len}(L) - 1$.

Donc la fonction `tri_rapide` termine.

Question 9

Montrez la correction de `tri_rapide`.

Cas de base

Si $\text{len}(L) \leq 1$, alors `L[:]` est bien triée.

Induction / Invariant

Supposons la fonction correcte pour les listes de longueur $< n$. Soit `L` de longueur n .

Alors :

- `plus_petits_triee` contient exactement les éléments plus petits ou égaux au `pivot` (sauf lui-même), et est triée.
- `plus_grands_triee` contient exactement les éléments strictement plus grands que le `pivot`, et est triée.

Donc `plus_petits_triee + [pivot] + plus_grands_triee` contient exactement les éléments de `L`, et est triée.

Donc, comme `tri_rapide` termine, elle est correcte.